

K L Deemed to be University

DATA STRUCTURES AND ALGORITHMS - 2

CO3 Java Code - Minimum Spanning Tree

Course Outcome	CO3
Topic	Minimum Spanning Tree using Kruskal and Prim
Language	Java
Document Type	Separate Implementation Code PDF

Objective

- Represent an undirected weighted graph using edge lists and adjacency lists.
- Implement Kruskal algorithm using Disjoint Set Union.
- Implement Prim algorithm using a priority queue/min-heap.

How to Run

Save the code in a file using the public class name shown in the program, compile it using javac, and run it using java.
Example: javac FileName.java followed by java FileName.

Complete Java Code

```
import java.util.*;

/*
 * CO3 Topic: Minimum Spanning Tree using Kruskal and Prim
 * This program implements both MST algorithms for an undirected weighted graph.
 */
public class CO3_MST_Demo {

    static class Edge implements Comparable<Edge> {
        int u, v, w;

        Edge(int u, int v, int w) {
            this.u = u;
            this.v = v;
            this.w = w;
        }

        public int compareTo(Edge other) {
            return this.w - other.w;
        }

        public String toString() {
            return name(u) + "-" + name(v) + "(" + w + ")";
        }
    }

    static class DSU {
        int[] parent;
        int[] rank;

        DSU(int n) {
            parent = new int[n];
            rank = new int[n];
            for (int i = 0; i < n; i++) parent[i] = i;
        }

        int find(int x) {
            if (parent[x] != x) parent[x] = find(parent[x]);
            return parent[x];
        }

        boolean union(int a, int b) {
            int pa = find(a);
            int pb = find(b);
            if (pa == pb) return false;

            if (rank[pa] < rank[pb]) parent[pa] = pb;
            else if (rank[pa] > rank[pb]) parent[pb] = pa;
            else {
                parent[pb] = pa;
                rank[pa]++;
            }
            return true;
        }
    }

    static String name(int index) {
        return String.valueOf((char)('A' + index));
    }

    static int kruskalMST(int vertices, List<Edge> edges) {
        Collections.sort(edges);
        DSU dsu = new DSU(vertices);
        int cost = 0;
        List<Edge> selected = new ArrayList<>();

        for (Edge e : edges) {
            if (dsu.union(e.u, e.v)) {
                selected.add(e);
                cost += e.w;
            }
        }

        System.out.println("Kruskal selected edges: " + selected);
        return cost;
    }

    static int primMST(int vertices, List<Edge>[] graph, int start) {
        boolean[] visited = new boolean[vertices];
        PriorityQueue<Edge> pq = new PriorityQueue<>();
        List<Edge> selected = new ArrayList<>();
        int cost = 0;

        // dummy edge allows the start vertex to enter the MST
    }
}
```

```

pq.add(new Edge(start, start, 0));

while (!pq.isEmpty() && selected.size() < vertices - 1) {
    Edge current = pq.poll();
    int node = current.v;

    if (visited[node]) continue;
    visited[node] = true;

    if (current.u != current.v) {
        selected.add(current);
        cost += current.w;
    }

    for (Edge next : graph[node]) {
        if (!visited[next.v]) pq.add(next);
    }
}

System.out.println("Prim selected edges: " + selected);
return cost;
}

static void addEdge(List<Edge> edges, List<Edge>[] graph,
                    int u, int v, int w) {
    Edge e = new Edge(u, v, w);
    edges.add(e);
    graph[u].add(new Edge(u, v, w));
    graph[v].add(new Edge(v, u, w));
}

public static void main(String[] args) {
    int vertices = 6; // A, B, C, D, E, F
    List<Edge> edges = new ArrayList<>();
    List<Edge>[] graph = new ArrayList[vertices];
    for (int i = 0; i < vertices; i++) graph[i] = new ArrayList<>();

    addEdge(edges, graph, 0, 1, 4); // A-B
    addEdge(edges, graph, 0, 2, 3); // A-C
    addEdge(edges, graph, 1, 2, 2); // B-C
    addEdge(edges, graph, 1, 3, 5); // B-D
    addEdge(edges, graph, 2, 3, 7); // C-D
    addEdge(edges, graph, 2, 4, 8); // C-E
    addEdge(edges, graph, 3, 4, 6); // D-E
    addEdge(edges, graph, 3, 5, 4); // D-F
    addEdge(edges, graph, 4, 5, 1); // E-F
    addEdge(edges, graph, 1, 4, 9); // B-E

    int kruskalCost = kruskalMST(vertices, new ArrayList<>(edges));
    System.out.println("Kruskal MST cost = " + kruskalCost);

    int primCost = primMST(vertices, graph, 0);
    System.out.println("Prim MST cost = " + primCost);
}
}

```

Expected Sample Output

```

Kruskal selected edges: [E-F(1), B-C(2), A-C(3), D-F(4), B-D(5)]
Kruskal MST cost = 15
Prim selected edges: [A-C(3), C-B(2), B-D(5), D-F(4), F-E(1)]
Prim MST cost = 15

```

Implementation Notes

- Kruskal sorts all edges and accepts only those that do not form a cycle.
- Prim grows one connected tree by repeatedly choosing the cheapest crossing edge.
- Both algorithms produce an MST when the graph is connected and undirected.